



Algorithm 1042: Sparse Precision Matrix Estimation with SQUIC

ARYAN EFTEKHARI, University of Lausanne, Lausanne, Switzerland

LISA GAEDKE-MERZHÄUSER and DIMOSTHENIS PASADAKIS, Università della Svizzera Italiana, Lugano, Switzerland

MATTHIAS BOLLHÖFER, TU Braunschweig, Braunschweig, Germany

SIMON SCHEIDEGGER, University of Lausanne, and Enterprise for Society (E4S), Ecublens, Switzerland

OLAF SCHENK, Università della Svizzera Italiana, Lugano, Switzerland

We present SQUIC, a fast and scalable package for sparse precision matrix estimation. The algorithm employs a second-order method to solve the ℓ_1 -regularized maximum likelihood problem, utilizing highly optimized linear algebra subroutines. In comparative tests using synthetic datasets, we demonstrate that SQUIC not only scales to datasets of up to a million random variables but also consistently delivers runtimes that are significantly faster than other well-established sparse precision matrix estimation packages. Furthermore, we showcase the application of the introduced package in classifying microarray gene expressions. We demonstrate that by utilizing a matrix form of the tuning parameter (also known as the regularization parameter), SQUIC can effectively incorporate prior information into the estimation procedure, resulting in improved application results with minimal computational overhead.

CCS Concepts: • **Mathematics of computing** → **Statistical software**; **Mathematical software performance**;

Additional Key Words and Phrases: Sparse precision matrix estimation, covariance matrix, matrix factorization, matrix inversion

ACM Reference format:

Aryan Eftekhari, Lisa Gaedke-Merzhäuser, Dimosthenis Pasadakis, Matthias Bollhöfer, Simon Scheidegger, and Olaf Schenk. 2024. Algorithm 1042: Sparse Precision Matrix Estimation with SQUIC. *ACM Trans. Math. Softw.* 50, 2, Article 13 (June 2024), 18 pages.

<https://doi.org/10.1145/3650108>

We would like to acknowledge the financial support of the joint DFG-470857344 and SNSF-204817 project, and the scientific support and HPC resources provided by the Erlangen National High Performance Computing Center (NHR@FAU) of the Friedrich-Alexander-Universität Erlangen-Nürnberg (FAU). NHR@FAU funding is provided by federal and Bavarian state authorities and by the German Research Foundation (DFG)-440719683. This work was also supported by a grant from the Swiss National Supercomputing Centre (CSCS) under project ID32191.

Authors' Contact Information: Aryan Eftekhari (Corresponding author), University of Lausanne, Internef, Lausanne, Switzerland; e-mail: aryan.eftekhari@unil.ch; Lisa Gaedke-Merzhäuser, Università della Svizzera Italiana, Via Giuseppe Buffi, Lugano, Switzerland; e-mail: lisa.gaedke.merzhaeuser@usi.ch; Dimosthenis Pasadakis, Università della Svizzera Italiana, Via Giuseppe Buffi, Lugano, Switzerland; e-mail: dimosthenis.pasadakis@usi.ch; Matthias Bollhöfer, TU Braunschweig, Universitätsplatz, Braunschweig, Germany; e-mail: m.bollhoefer@tu-bs.de; Simon Scheidegger, University of Lausanne, and Enterprise for Society (E4S), Internef, Lausanne, Switzerland; e-mail: simon.scheidegger@unil.ch; Olaf Schenk, Università della Svizzera Italiana, Via Giuseppe Buffi, Lugano, Switzerland; e-mail: olaf.schenk@usi.ch.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7295/2024/6-ART13

<https://doi.org/10.1145/3650108>

1 Introduction

Estimating the inverse of the covariance matrix, referred to as the precision matrix, is a central topic in multivariate analysis with a wide range of applications; for example, the **linear discriminant analysis (LDA)** method for classification [5], portfolio optimization for determining the optimal weights for asset allocation [25], and the study of relationships among a set of genes [7]. Furthermore, it plays a fundamental role in the definition of Gaussian Markov Random Fields, which are utilized in a multitude of problems (see, e.g., [48] for recent software developments and [45] for a comprehensive theoretical review). A zero entry in the precision matrix represents the conditional independence between the associated pair of random variables. In high-dimensional datasets with a large number of random variables, a common assumption is that the estimated precision matrix will have some degree of sparsity, that is, many random variables will be, or at least will be approximated as, conditionally independent (see, e.g., applications in biological networks [32, 52], finance [20, 34], and pattern recognition [28, 38]).

The ever-increasing trend of high-dimensional datasets, with hundreds of thousands to millions of random variables, has emphasized the importance of scalable methods for sparse precision matrix estimation [21]. The ℓ_1 -regularized **maximum likelihood (ML)** method is a common and widely used approach for estimating sparse precision matrices [3, 22, 44, 53]. There is a plethora of optimization strategies for tackling the regularized likelihood problem. Notably, second-order methods provide superior convergence rates but have a higher computational cost [30, 39]. In particular, the **QUadratic approximation of Inverse Covariance matrix (QUIC)** [26] is a computationally efficient Newton-like method, intended for relatively small datasets with hundreds to thousands of random variables. BigQUIC [27] addresses the primary computational bottlenecks of QUIC, which lie in the representation and inversion of dense matrices. It tackles these challenges by evaluating entries of large dense matrices on an as-needed basis and by performing approximate matrix inversion. However, the runtimes of BigQUIC can become impractical (surpassing several hours) for modestly sized problems with tens of thousands of random variables, making the algorithm somewhat prohibitive for high-dimensional or time-sensitive applications (see, e.g., the time-sensitive case study presented in [19]). The **Sparse QUIC (SQUIC)** [6] algorithm is an extension of these two algorithms that emphasizes the use of sparse representations and approximations of otherwise dense matrices. The algorithm was enhanced in [16] by introducing distributed- and shared-memory parallelism to improve scalability. Although it is appropriate to assume that the precision matrix is sparse, intermediate computations may not be sparse themselves. For high-dimensional distributions, this results in a considerably higher computational load. As described in [19], reduced sparsity in the intermediary computation is a prevalent scenario in real-world applications, which motivated the use of block-oriented computational techniques to increase cache efficiency. In scenarios of reduced sparsity, the blocking variant of the SQUIC algorithm has proven to be highly effective in decreasing runtimes when compared to the non-blocking SQUIC algorithm.

The contributions of this article are threefold. First, we introduce the SQUIC software package (differentiated from the algorithm denoted as SQUIC), which is the culmination of the preceding work as well as an extension of the QUIC and BigQUIC algorithms. This codebase is the first complete implementation of the experimental development in [6], which incorporates shared-memory parallelism as outlined in [16] and the blocking algorithmic operations as detailed in [19]. Second, we incorporate a scalable *matrix tuning parameter* feature (also called the regularization parameter) that allows practitioners to enforce a prior bias in the estimation procedure. This enables each element of the precision matrix to be penalized individually, allowing one to enforce, if available, additional information about the nonzero structure of the precision matrix. Third,

we provide comparative test and case studies based on synthetic and real-world datasets that highlight the basic usage of the package as well as demonstrate the effectiveness of the matrix tuning parameter. In particular, we conducted comparative tests between SQUIC, glasso [22, 23], EQUAL [49, 50], BigQUIC [27, 33], and SNETCH [29] to evaluate the estimation accuracy and the runtime performance for datasets of up to a million random variables. Additionally, we provide examples of the usage of the matrix tuning parameter in both synthetic and real-world scenarios, showcasing how it can enhance estimation accuracy and, consequently, improve application results. The package is implemented in C++, parallelized using OpenMP [10], and utilizes high-performance libraries such as CHOLMOD [9]. The SQUIC package is structured around a library with interfaces for R and Python. The library can be compiled from the source code or downloaded directly as precompiled versions.

The remainder of the article is organized as follows: Section 2 recaps the estimation of precision matrices via an ℓ_1 -regularized optimization problem. Section 3 outlines the key components of the software package. Then, in Section 4, we present numerical experiments using both synthetic and real-world datasets, and in Section 5 we conclude.

2 Background

Consider $\mathbf{Y} \in \mathbb{R}^{p \times n}$, representing n independent samples from a p -variate Gaussian distribution, $\mathcal{N}(\boldsymbol{\mu}^*, \Sigma^*)$. Here, $\Sigma^* \in \mathbb{R}^{p \times p}$ and $\boldsymbol{\mu}^* \in \mathbb{R}^p$ denote the true covariance matrix and mean, with their sample estimates being

$$\hat{\boldsymbol{\mu}}_i = \frac{1}{n} \sum_{m=1}^n \mathbf{Y}_{im}, \text{ and } \mathbf{S}_{ij} = \frac{1}{n} \sum_{m=1}^n (\mathbf{Y}_{im} - \hat{\boldsymbol{\mu}}_i)(\mathbf{Y}_{jm} - \hat{\boldsymbol{\mu}}_j), \quad (1)$$

respectively. The ML method is an approach for recovering an estimate of the precision matrix $\boldsymbol{\Theta} := \Sigma^{-1}$, which can be expressed as the minimization of the negative log-likelihood function

$$g(\boldsymbol{\Theta}) = -\log \det \boldsymbol{\Theta} + \text{tr}[\mathbf{S}\boldsymbol{\Theta}]. \quad (2)$$

When $p \leq n$ and $\mathbf{S}$ is nonsingular, the ML estimator of $\boldsymbol{\Theta}$ is $\mathbf{S}^{-1}$. However, in the more common scenario when $p \gg n$, the inverse $\mathbf{S}^{-1}$ does not exist. The ℓ_1 -regularization of Equation (2) allows for a well-defined optimization problem which also induces sparsity in the estimated precision matrix. From a computational perspective, the induced sparsity is critical as it opens the door to research in high-performance algorithms for sparse linear systems (see, e.g., [12] for a survey dating back 50 years). Formally, the ℓ_1 -regularized ML estimator reads

$$\hat{\boldsymbol{\Theta}} = \underset{\boldsymbol{\Theta} \succ 0}{\text{argmin}} \left\{ g(\boldsymbol{\Theta}) + \|\boldsymbol{\Lambda} \odot \boldsymbol{\Theta}\|_1 \right\}, \quad (3)$$

where $\boldsymbol{\Theta} \succ 0$ denotes positive-definiteness and $\boldsymbol{\Lambda} \in \mathbb{R}^{p \times p}$ is the matrix tuning parameter which defines the symmetric and positive penalty coefficients $\Lambda_{ij} = \Lambda_{ji} > 0$. Here we denote $\odot$ as the element-wise matrix product $[\boldsymbol{\Theta} \odot \boldsymbol{\Lambda}]_{ij} = \Theta_{ij}\Lambda_{ij}$. The proximal Newton-type method, a second-order approach, offers an efficient solution to Equation (3) (see, e.g., [35] for description). This strategy focuses on the iterative optimization of an ℓ_1 -regularized local quadratic approximation of g . At a given iteration, for a certain perturbation Δ , the second-order approximation of $g(\boldsymbol{\Theta} + \Delta)$ up to a constant term is

$$\hat{g}(\Delta) := \text{tr}[(\mathbf{S} - \boldsymbol{\Theta}^{-1})\Delta] + \frac{1}{2} \text{tr}[\boldsymbol{\Theta}^{-1}\Delta\boldsymbol{\Theta}^{-1}\Delta]. \quad (4)$$

Rather than directly minimizing the objective function in Equation (3), we successively minimize the local quadratic model of the objective and compute iterative estimates for the solution along the Newton direction

$$\mathbf{U} = \underset{\Delta}{\operatorname{argmin}} \left\{ \hat{g}(\Delta) + \|\Lambda \odot (\Theta + \Delta)\|_1 \right\}. \quad (5)$$

Using a line-search procedure with an appropriate step-size $\alpha \in (0, 1]$ and direction $\mathbf{U}$, we can compute the subsequent estimate of the solution as $\Theta + \alpha\mathbf{U}$. The choice of α must satisfy both that $\Theta + \alpha\mathbf{U} \succ 0$ and also the Armijo-rule condition. We delve into this topic further in Section 3.

The QUIC algorithm, a proximal Newton-type method, serves as the foundation of the SQUIC algorithm. We highlight two key contributions of the QUIC algorithm for conciseness (for further details, we refer the reader to [26]). The first is the derivation of a closed-form solution to Equation (5) using a coordinate descent update rule. The *block-based* variant of the coordinate descent update will be detailed in Section 3. The second key contribution is a variable selection strategy that uses the stationary conditions of Equation (3) to perform updates only on a subset of elements $\mathbf{U}_{ij}$, and in turn, of Θ_{ij} . The indices updated during this process are

$$\mathcal{I}_{\text{free}} := \left\{ (i, j) : \Lambda_{ij} \leq |\mathbf{S}_{ij} - [\Theta^{-1}]_{ij}| \text{ or } \Theta_{ij} \neq 0 \right\}, \quad (6)$$

and the remaining indices are *fixed* to a value of zero, satisfying the condition $\Theta_{ij} = 0, \forall (i, j) \notin \mathcal{I}_{\text{free}}$. The SQUIC algorithm discussed next builds on these key algorithmic developments with a focus on performance and scalability.

3 Second-Order Sparse Precision Matrix Estimation

The SQUIC algorithm is a second-order method that mitigates computational challenges associated with large-scale sparse precision matrix estimation, particularly with dense matrix representations and $O(p^3)$ complexity operations. It achieves this by optimizing the core operations to leverage the sparsity and parallelism inherent in the problem, thereby enhancing performance and scalability. We begin by presenting the algorithmic description of SQUIC, followed by a discussion of computationally demanding operations and the targeted solutions implemented to address these challenges, and conclude with a high-level overview of the implemented parallelization strategy.

3.1 Algorithmic Description

The computational steps for SQUIC are shown in Algorithm 1. Notice, that the key differentiating factors between the SQUIC and QUIC algorithms are the operations outlined in Section 3.2. The inputs of the algorithm are $\mathbf{S}, \Lambda, T, \tau, \Theta$, and Θ^{-1} , corresponding to the sample covariance matrix, matrix tuning parameter, maximum number of iterations, convergence tolerance, and the initial guess of the estimated precision matrix and its inverse, respectively. The initial guess for the precision matrix, and its corresponding inverse, can be any sparse symmetric positive-definite matrix; in most cases, this will be identity. Upon entering the iterative portion of the algorithm, we compute the objective function for the current value of Θ in step 2. Following this, in step 3, using Θ^{-1} (either from the initial guess or obtained from step 15), we compute $\mathcal{I}_{\text{free}}$ as defined in Equation (6). In step 4, the Newton direction $\mathbf{U}_{ij}$ for indices in $\mathcal{I}_{\text{free}}$ is computed using the coordinate descent update, as shown in Equation (5). Steps 5–11 focus on the line-search procedure, which involves an update to the current iterate. Here, we select a step-size α that assures the updated iterate is positive-definite (verified using Cholesky factorization) and satisfies the Armijo-rule condition, denoted as AC. For further details on the Armijo-rule condition, we refer the reader to section 3.2 in [26]. In steps 12–14, the algorithm checks for convergence by computing the objective function

Algorithm 1 SQUIC/QUIC

Require: $\mathbf{S}, \Lambda, T, \tau, \Theta, \Theta^{-1}$

```

1: for  $t = 1$  to  $T$  do
2:    $\text{obj} \leftarrow g(\Theta) + \|\Lambda \odot \Theta\|_1$ 
3:   compute :  $\mathcal{I}_{\text{free}}$ 
4:   compute :  $\mathbf{U}_{ij} \forall (i, j) \in \mathcal{I}_{\text{free}}$ 
5:   for  $r = 0$  to  $20$  do
6:      $\alpha \leftarrow 0.5^r$ 
7:     if  $\Theta + \alpha \mathbf{U} \succ 0$  and  $\text{AC}(\Theta, \mathbf{U}, \alpha, \Lambda)$  then
8:        $\Theta \leftarrow \Theta + \alpha \mathbf{U}$ 
9:       break
10:    end if
11:  end for
12:  if  $|\text{obj} - (g(\Theta) + \|\Lambda \odot \Theta\|_1)|/|\text{obj}| < \tau$  then
13:    break
14:  end if
15:   $\Theta^{-1} \leftarrow \text{inv}(\Theta)$ 
16: end for
17: return  $\Theta, \Theta^{-1}$ 

```

at the updated Θ . Last, in step 15, using the Cholesky factors computed in step 7, we compute the matrix inverse to be used in the next iteration. Finally, steps 2–16 are repeated until convergence.

3.2 Leveraging Sparsity

The SQUIC algorithm addresses five key computational challenges, starting with (1) the sample covariance matrix $\mathbf{S}$. Subsequently, within each iteration, the following computationally demanding operations are performed: (2) the Newton direction $\mathbf{U}$ used in the coordinate descent update, (3) the Cholesky factorization used, alongside other computations, to verify the positive-definiteness of the updated Θ , and finally, (4) the matrix inversion to evaluate Θ^{-1} . Furthermore, the representation of the matrix tuning parameter Λ poses challenges for large-scale applications due to its inherent (5) dense matrix representation. In this section, we begin by presenting the SQUIC implementation of the matrix tuning parameter, specifically tailored for large-scale applications. We then proceed to outline the key computational operations previously developed in [6, 19], encompassing the sparse computation of the sample covariance matrix, the supernodal sparse Cholesky factorization, the block approximate matrix inversion, and last, the block coordinate descent update.

3.2.1 Matrix Tuning Parameter. The matrix tuning parameter Λ is a dense matrix, which, due to computational constraints, is impractical to represent explicitly when p is large. To bypass this issue, we represent Λ using a sparse, non-negative symmetric matrix $\mathbf{M} \in \mathbb{R}^{p \times p}$, in conjunction with a *scalar tuning parameter* $\lambda \in \mathbb{R} > 0$, as follows:

$$\Lambda_{ij} := \begin{cases} \mathbf{M}_{ij}, & \text{if } \mathbf{M}_{ij} \neq 0 \\ \lambda & \text{otherwise.} \end{cases} \quad (7)$$

Given the expectation that the estimated precision matrix will be sparse, we can effectively leverage the nonzero pattern of $\mathbf{M}$ to encode a penalty for the anticipated nonzero entries, while employing λ for all other entries. Using this representation, we can effectively express the p^2 values of Λ with

$\text{nnz}(\mathbf{M}) + 1$ values. Notably, if we set $\mathbf{M} = \mathbf{0}$, then $\Lambda_{ij} = \lambda$, reverting to the conventional penalty formulation of the regularization term in Equation (3), which is $\lambda \|\Theta\|_1$. This basic construction provides a means to encode entry-wise specific values of Λ while still circumventing scalability issues for large-scale applications. While glasso and QUIC offer similar functionality, they utilize dense representations and are not suitable for large-scale datasets (see Section 4.2 for a real-world application using the matrix tuning parameter).

3.2.2 Sparse Sample Covariance Matrix. The sample covariance matrix $\mathbf{S}$ is dense, leading to significant if not infeasible memory requirements for large p . Therefore, we compute a sparse representation of $\mathbf{S}$, wherein all diagonal and only the large off-diagonal elements are retained. Formally, we preserve the entries in $\mathbf{S}$ that satisfy

$$\Lambda_{ij} \leq |\mathbf{S}_{ij}| \quad \forall (i, j), \text{ or } i = j. \quad (8)$$

Given that the diagonal elements are always computed, we conduct an initial, cost-efficient check: if the condition $\sqrt{\mathbf{S}_{ii}\mathbf{S}_{jj}} \leq \Lambda_{ij}$ holds, it immediately eliminates the need to compute the entry $\mathbf{S}_{ij}$, as then $|\mathbf{S}_{ij}| \leq \sqrt{\mathbf{S}_{ii}\mathbf{S}_{jj}} \leq \Lambda_{ij}$. Notice that $\mathbf{S}$ is required in the evaluation of the free index-set $\mathcal{I}_{\text{free}}$ in Equation (6), which in turn requires $\mathbf{S}$ to be computed for all nonzero entries of Θ^{-1} . Under regular circumstances, all elements of $\mathbf{S}$ would be computed as Θ^{-1} is dense. However, as we will discuss in Section 3.2.4, we utilize a sparse approximation of Θ^{-1} , and thus it suffices to ensure that the sparsity pattern of $\mathbf{S}$ overlaps with that of the sparse approximation of Θ^{-1} . Therefore, in each iteration we compute any missing values of $\mathbf{S}$ on the fly such that for every nonzero entry in the approximated Θ^{-1} a corresponding entry exists in $\mathbf{S}$. This approach of constructing $\mathbf{S}$ is not an approximation but rather removes redundancy (see, e.g., Section 3.6 of [15] for further details). From a computational standpoint, it achieves a balance between the cache efficiency of dense matrix multiplication and the memory efficiency of computing $\mathbf{S}$ on the fly, or as required in Equation (6). Unlike SQUIC, BigQUIC exclusively computes the entries of $\mathbf{S}$ on the fly, trading cache efficiency for a reduced memory footprint.

3.2.3 Supernodal Sparse Cholesky Factorization. Cholesky factorization is required for the line-search procedure, in which we seek the largest step-size α , such that the updated iterate is positive-definite and sufficiently reduces the objective function (see, e.g., steps 5–11 in Algorithm 1 for details). As the estimated precision matrix is sparse, we employ the high-performance supernodal sparse Cholesky factorization routine, CHOLMOD [9], to efficiently leverage this sparsity. Here, block-oriented data structures known as *supernodes* are employed in the factorization routine and subsequently repurposed in the block sparse approximate matrix inversion component of the algorithm (discussed in the next section). For the Cholesky factor $\mathbf{L}$, a supernode is the maximal block of contiguous columns of $\mathbf{L}$ with a dense lower triangular diagonal block and an identical sparsity pattern for the off-diagonal block column $\mathbf{L}_{:,b}$ (see, e.g., [37] for further details). A visual representation of the data structures is shown in Figure 1. The Cholesky, or similarly the LDL, decomposition is defined as

$$\Theta = \mathbf{P}\mathbf{L}\mathbf{D}\mathbf{L}^T\mathbf{P}^T, \quad (9)$$

where $\mathbf{P}$ is the fill-in reducing permutation matrix, $\mathbf{D}$ is a block-diagonal matrix with dense positive-definite diagonal blocks, and $\mathbf{L}$ is a lower triangular block matrix. The supernodal block structure, along with the decomposition in Equation (9), is later used to perform an approximate matrix inversion using the block structures from the supernodal approach. Furthermore, the log-determinant, required for computing the objective function, can be evaluated using dense matrix operations on much smaller block diagonals of $\mathbf{D}$ [19]. In contrast, the line-search procedure of BigQUIC does not use Cholesky factorization. It evaluates positive-definiteness by computing the

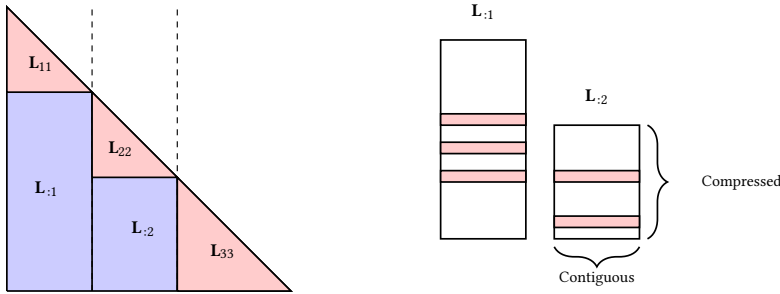


Fig. 1. A visual representation of supernodal LDL (left) for $\Theta = \mathbf{LDL}^\top$ where $\mathbf{D}$ is block diagonal with blocks $b = 1, 2, 3$, and $\mathbf{D}_{bb} = \mathbf{L}_{bb}\mathbf{L}_{bb}^\top$. Elements colored in red, blue, and white are dense, sparse, and zero respectively. The $\mathbf{L}_{:,b}$ block is stored with compressed rows and contiguous columns (right).

log-determinant of the updated precision matrix directly in a recursive fashion. This method relies on computing the inverse of Θ , which, like SQUIC, it approximates—a topic we will discuss in the next section.

3.2.4 Block Sparse Approximate Matrix Inversion. Matrix inversion is critical to the algorithm and presents a major computational challenge. Using the decomposition in Equation (9), we can efficiently compute $\Theta^{-1} = \mathbf{L}^{-\top}\mathbf{D}^{-1}\mathbf{L}^{-1}$ for datasets with a smaller p , which denotes lower dimensionality. However, exact inversion becomes problematic for high-dimensional datasets as $\mathbf{L}^{-1}$ is dense, rendering the evaluation of $\mathbf{L}^{-1}$ and that of $\mathbf{L}^{-\top}\mathbf{D}^{-1}\mathbf{L}^{-1}$ infeasible for large p . Here, we approximate the exact inverse of $\mathbf{L}$ using a truncated and thresholded Neumann series matrix inversion scheme. This sparse approximation is denoted as $\mathbf{L}^{\text{inv}} \approx \mathbf{L}^{-1}$. Let $\mathbf{L} = \mathbf{I} - \mathbf{E}$, where $-\mathbf{E}$ refers to the strictly lower triangular part of $\mathbf{L}$, the sparse approximation is computed as

$$\mathbf{L}_{k+1}^{\text{inv}} = \mathbf{L}_k^{\text{inv}}\mathbf{E} + \mathbf{I}, \quad k = 1, 2, \dots, p-1, \quad (10)$$

where $\mathbf{L}_1^{\text{inv}} := \mathbf{I} + \mathbf{E}$. As discussed in [19], the approximate matrix inversion method may encounter scenarios where $\mathbf{L}_{k+1}^{\text{inv}}$ exhibits reduced sparsity. In these instances, taking advantage of the block structure inherent in the supernodal Cholesky factorization has proven beneficial in maintaining an efficient computational scheme, even when $\mathbf{L}^{\text{inv}}$ contains thousands of nonzeros per row. Given an *inversion tolerance* $\tau_{\text{inv}} > 0$, the successive summation in Equation (10) is truncated when $|\mathbf{L}_{k+1}^{\text{inv}} - \mathbf{L}_k^{\text{inv}}|_{ij} \leq \tau_{\text{inv}}$ is fulfilled for every (i, j) . Furthermore, for $\gamma \in (0, 1)$, the update values are thresholded such that only values for which $|\mathbf{L}_{k+1}^{\text{inv}} - \mathbf{L}_k^{\text{inv}}|_{ij} > \gamma \tau_{\text{inv}}$ are updated. In the provided implementation, γ is set to 10^{-1} . Upon the convergence of Equation (10), we utilize the decomposition of Equation (9), substituting $\mathbf{L}^{-1}$ with $\mathbf{L}^{\text{inv}}$, to compute the sparse approximate inverse, Θ^{inv} . Note that the inversion operation for $\mathbf{D}$ is computationally inexpensive, due to its block-diagonal structure. Each diagonal block can be straightforwardly computed using LAPACK (see Figure 1 for visualization). Finally, to keep Θ^{inv} sparse, we only retain elements that satisfy $[\Theta^{\text{inv}}]_{ij}^2 > \tau_{\text{inv}}^2 [\Theta^{\text{inv}}]_{ii} [\Theta^{\text{inv}}]_{jj}$, discarding the rest from Θ^{inv} . In Section 4.1.1, we present test results which highlight the effectiveness of this approach. Similar to SQUIC, BigQUIC also approximates Θ^{-1} . However, it approximates the columns of Θ^{-1} as required. This is essentially done by solving the linear system $\Theta\Theta^{-1} = \mathbf{I}$ using the conjugate gradient method.

3.2.5 Block Coordinate Descent Update. Given the current Newton direction $\mathbf{U}$, as defined in Equation (5), we compute the new Newton direction using $\mathbf{U}'_{ij} \leftarrow \mathbf{U}_{ij} + \mu$ for all $(i, j) \in \mathcal{I}_{\text{free}}$, where μ is the value updated by the coordinate descent. This *sweep* of the coordinate descent update is

performed multiple times per iteration. For ease of notation, we denote $\mathbf{W} := \Theta^{\text{inv}}$. The closed-form solution of the coordinate decent update is as follows:

$$\begin{aligned} \mu &= -\mathbf{C}_{ij} + \text{ST}(\mathbf{C}_{ij} - \mathbf{B}_{ij}/\mathbf{A}_{ij}, \Lambda_{ij}/\mathbf{A}_{ij}), \text{ where,} \\ \mathbf{A}_{ij} &:= \mathbf{W}_{ij} + \mathbf{W}_{ii}\mathbf{W}_{jj}, \quad \mathbf{B}_{ij} := \mathbf{S}_{ij} - \mathbf{W}_{ij} + \mathbf{W}_{i:}\mathbf{U}\mathbf{W}_{:j}, \text{ and } \mathbf{C}_{ij} := \Theta_{ij} + \mathbf{U}_{ij}. \end{aligned} \quad (11)$$

Here $\text{ST}(a, b) := \text{sign}(a) \max(|a| - b, 0)$ is the soft thresholding function, with $\mathbf{W}_{i:}$ and $\mathbf{W}_{:j}$ denoting the i th row and j th column of $\mathbf{W}$, respectively (see, e.g., [26] for the derivation of Equation (11)). The blocking strategy deployed here is designed to boost cache efficiency in the computation of the coordinate descent update by eliminating redundant memory access and improving contiguous access patterns (see, e.g., [19] for details). This is achieved by grouping the indices $(i, j) \in \mathcal{I}_{\text{free}}$ into blocks of updates sharing the same column index, specifically formatted as $\{(i_1, j), (i_2, j), (i_3, j) \dots\}$. Within each column block, the row indices are organized in ascending order, following $i_1 < i_2 < i_3 \dots$. This arrangement optimizes the iteration over $(i, j) \in \mathcal{I}_{\text{free}}$ by enabling efficient, one-time access to the columns of $\mathbf{S}$, $\mathbf{W}$, and Θ . Furthermore, within each column block, we have increasing row indices, thereby providing a contiguous access pattern for the updates to the entries of $\mathbf{U}'_{:j}$. Similarly, BigQUIC also adopts a blockwise approach for the coordinate descent update, albeit utilizing a different technique. In their implementation, blocks are defined using the partitioning algorithm METIS [31].

3.3 Parallelization

The proposed package relies on OpenMP for parallelization. The operations that are parallelized in the algorithm include the computation of the sparse sample covariance matrix, the sparse Cholesky factorization, and the sparse approximate inversion, as described in Sections 3.2.2, 3.2.3, and 3.2.4, respectively. In the sample covariance matrix computation, assuming the data has a zero mean such that $\mathbf{S} = \mathbf{Y}\mathbf{Y}^T/n$, the parallel compute resources are evenly distributed along the rows of $\mathbf{Y}$ to perform the matrix multiplication. A similar approach is employed in the approximate matrix inversion, where the parallelization takes place along the block columns of $\mathbf{L}$ (as shown in Figure 1). These block columns are derived from the supernodal Cholesky factorization routine, which also employs shared-memory parallelization internally. The parallel efficiency of the different components has been extensively documented in [16, 19].

4 Numerical Experiments

In this section, we highlight the numerical results from our developed SQUIC package. The package is implemented in C++ and parallelized using OpenMP for both the sparse sample covariance matrix and the block approximation matrix inversion computations. For further details on the parallelization and results, we refer the reader to [16, 19]. In Section 4.1, we will conduct a set of tests on synthetic datasets to highlight the performance and accuracy of the SQUIC package. Following this, Section 4.2 will showcase an example application, particularly for the LDA of large microarray datasets with a limited number of samples.

All experiments presented here were conducted in R on an AMD EPYC 7402 2.8 GHz (24 cores) workstation equipped with 1TB of memory. For further details on the installation procedure of the SQUIC package, including the source code, the accompanying interfaces, and all replication scripts for the experiments outlined, please refer to [17].

4.1 Tests with Synthetic Data

For standardized comparisons, we generate two test cases of synthetic datasets with p -variate Gaussian random variables, using the following predefined precision matrices:

- **Tridiagonal:** A tridiagonal matrix with off-diagonal values of -0.5 and 1.25 on the diagonal.
- **Clusters:** A random structured matrix representing a graphical structure of $p/100$ clusters of size 100, and an average degree of 20 with 90% of the edges contained within the clusters [2].

The tridiagonal test case represents a simple, yet frequently used, example where the underlying random variables exhibit chain-like dependencies. On the other hand, the clusters dataset serves as a reliable surrogate for representing typical observations of clustered dependencies, as documented in [19, 40], and also used in numerical tests in [27]. A distinctive characteristic of these clustered dependencies, from the computational standpoint of the SQUIC algorithm, is that intermediate computations, such as Cholesky factorization and matrix inversion, become challenging due to the reduced sparsity. However, these performance issues are mitigated through the implementation of block-oriented computations, as outlined in Section 4.1.¹

In all the tests we have fixed the number of samples to $n = 500$. The dataset $\mathbf{Y}$ is computed by solving the linear system $\mathbf{Y}\mathbf{L} = \mathbf{R}$, where $\mathbf{R} \sim \mathcal{N}(0, \mathbf{I}) \in \mathbb{R}^{p \times n}$ and $\mathbf{L}$ is the lower-triangular Cholesky factor of the exact precision matrices discussed earlier. Due to performance limitations within the R environment, the dataset generation process is performed externally, and the datasets are provided alongside the replication scripts. Where applicable, we provide results for comparable R packages for sparse precision matrix estimation, namely *glasso* [23, 22], *BigQUIC* [33, 27], and *EQUAL* [49, 50]. In addition, we provide a comparison with the Matlab-based *SNETCH* [29] package, which, similar to the proposed SQUIC package, utilizes sparse Cholesky factorization for fast sparse precision matrix estimation. For all tests and packages used, we normalize the datasets such that the sample covariance matrix $\mathbf{S}$ satisfies $\mathbf{S}_{ij} \in [-1, 1]$ with $\mathbf{S}_{ii} = 1$. This normalization is done to simplify the selection of an appropriate tuning parameter.

The experiments presented in the subsequent sections follow a logical progression. In Section 4.1.1, we validate the sparse approximate matrix inversion and determine a suitable inversion tolerance τ_{inv} . Utilizing this information, we assess accuracy metrics for the scalar and matrix tuning parameters in Sections 4.1.2 and 4.1.3, respectively. Finally, utilizing an appropriate tuning parameter, we present large-scale performance results in Section 4.1.4.

4.1.1 Sparse Approximate Matrix Inversion. This test highlights the attributes of the block sparse approximate matrix inversion scheme (see Section 3.2.4 for details). The experiments are conducted using both the tridiagonal and clusters datasets, each of size $p = 2 \cdot 10^3$. The impact of the inversion tolerance τ_{inv} on both the number of nonzero elements per row of the inverse and the approximation error is depicted in the left and right panels of Figure 2, respectively. Here, we define the inversion error as $\|\Theta\Theta^{\text{inv}} - \mathbf{I}\|_{\text{max}}$. For these tests, we use $\lambda = 0.2$ for both datasets. For these datasets, Θ contains between 3 and 5 nonzeros per row, and its exact inverse would not be sparse. For the tridiagonal example, we can see that it is feasible to compute an extremely sparse approximation of the inverse while maintaining minimal error, even under notably strict tolerances. This can be attributed to the fact that the absolute value of the off-diagonal entries in the exact inverse decays exponentially as they move away from the diagonal. However, the tests conducted on the clusters dataset demonstrate a decrease in sparsity within the approximate inversion. This phenomenon was acknowledged and addressed in [19], through the introduction of the *block* version of the sparse matrix inversion algorithm, which enhanced the runtime performance of the package in such scenarios. Nonetheless, even with a relatively strict tolerance of 10^{-4} , Θ^{inv} has 50 nonzeros per row. From our experiments, we have observed that the performance of the SQUIC package starts to diminish when the approximated inverse of the precision matrix contains more than 500 nonzeros

¹For further performance comparisons between an experimental development of the SQUIC algorithm and other publicly available packages, we refer the reader to [6, 16, 19].

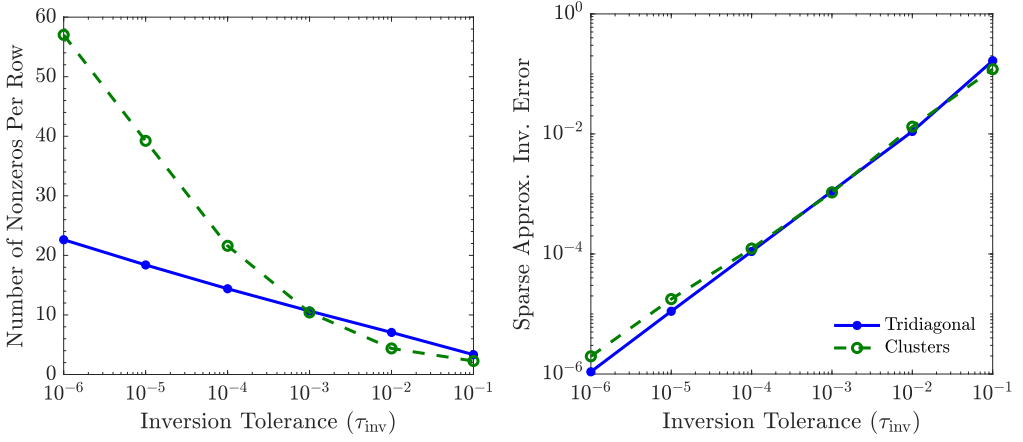


Fig. 2. An analysis of the sparse approximate matrix inversion method, as outlined in Section 3.2.4, when applied to the tridiagonal and clusters datasets for a matrix size of $p = 2 \cdot 10^3$. The (left) number of nonzeros per row and the (right) matrix inversion error (defined here as $\|\Theta\Theta^{\text{inv}} - \mathbf{I}\|_{\max}$) are shown with respect to the inversion tolerance τ_{inv} .

per row. At this stage, the performance decline is not solely attributed to the sparse approximate matrix inversion but is also compounded by subsequent computations in the iterations, which include Cholesky factorization and coordinate descent updates.

In applications where high convergence of the objective is not required, using a relatively large inversion tolerance can be advantageous as it reduces the runtime. However, if the inversion tolerance is selected to be too large, it could degrade the second-order convergence rate, as the Newton direction would no longer be accurately represented. In most applications, an inversion tolerance level in the range of 10^{-3} to 10^{-4} is sufficient, and enforcing stricter tolerance levels does not lead to material differences in the estimate. For the following numerical experiments, we fix $\tau_{\text{inv}} = 10^{-4}$ and use it as the default value of the package.

4.1.2 Accuracy with Scalar Tuning Parameter. In this test we study the accuracy of the estimated nonzero structure of the precision matrix $\hat{\Theta}$. Let s be the number of nonzeros in the off-diagonals of the true precision matrix Θ^* . The accuracy metric we will adopt to assess the retrieved nonzero structure is the

$$\text{F1-Score} := [1 + 0.5(\text{FP} + \text{FN}) / (s - \text{FN})]^{-1} \in [0, 1], \quad (12)$$

where FN (false negatives) is the number of entries in $\hat{\Theta}$ that are incorrectly estimated to be zero and FP (false positives) represents the number of entries in $\hat{\Theta}$ that are incorrectly estimated to be nonzero.² A value of F1-Score = 1 indicates a perfect retrieval of the off-diagonal nonzero structure (see, e.g., [11] for further details on the F1-Score accuracy metric). It should be noted that the selection of the tuning parameter, whether in the scalar or matrix form, is problem-dependent. There exists a variety of established techniques, including **K-fold cross-validation (K-CV)** [14], **Akaike Information Criterion (AIC)** [1], **Bayesian Information Criterion (BIC)** [46], and the

²Given the symmetry and the constant presence of diagonal entries, we focus solely on the strictly upper triangular part of $\hat{\Theta}$. Moreover, the formulation in Equation (12) uses the equality $\text{FN} + \text{TP} = s$, where TP (true positives) denotes the number of nonzero entries in $\hat{\Theta}$ that are correctly estimated as nonzero.

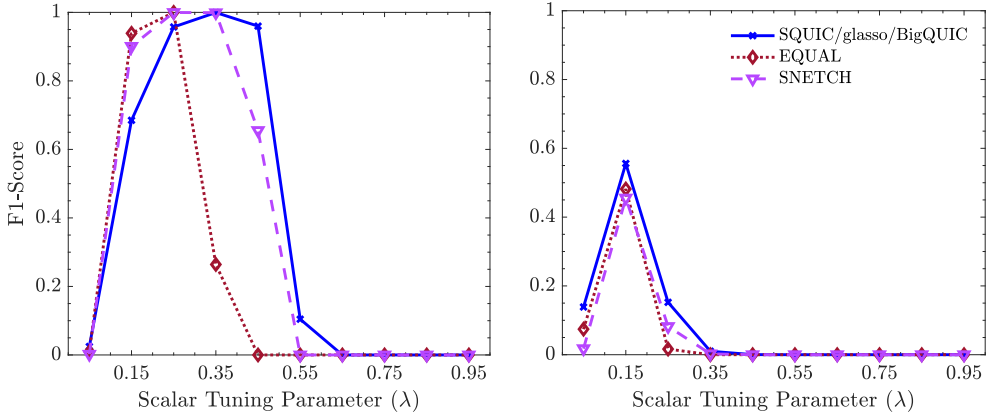


Fig. 3. A comparison of the F1-Score defined in Equation (12) for different precision matrix estimation packages with respect to the scalar tuning parameters λ for the (left) tridiagonal and (right) clusters dataset. The estimates using SQUIC, glasso, and BigQUIC produce the same F1-Score. Note that the tuning parameter for SNETCH is scaled by a factor of 25, meaning $\lambda_{\text{SNETCH}} = 25 \cdot \lambda$.

Stability Approach to Regularization Selection (StARS) [36], which can be applied to select the tuning parameter.

In Figure 3, we show the F1-Score with respect to a path of scalar tuning parameters λ for the tridiagonal and clusters datasets, shown in the left and right panels, respectively. We conducted both sets of tests on matrices of size $p = 2 \cdot 10^3$. The packages glasso, BigQUIC, and SQUIC report very similar accuracies in their recoveries, which is an expected behavior since they minimize the same objective function outlined in Equation (3), albeit using different computational approaches. Although SNETCH minimizes the same objective function as SQUIC, it estimates the sparse Cholesky factor $\mathbf{L}$, such that $\mathbf{L}\mathbf{L}^\top = \hat{\Theta}$, and as a result, the F1-Score differs compared to the aforementioned packages. The tuning parameter for SNETCH is scaled by a factor of 25, meaning $\lambda_{\text{SNETCH}} = 25 \cdot \lambda$. Overall, the peak F1-Score across the different packages is approximately 1 and 0.65 for the tridiagonal and clusters cases, respectively. In contrast, EQUAL minimizes a different objective function (see, e.g., [50] and references therein for details). Here we see similar accuracy trajectories and slightly higher peak accuracies in the clusters dataset. In the subsequent performance tests, we adopt the following standardized approach: for the tridiagonal test, we use $\lambda = 0.35$ for $p < 10^5$ and $\lambda = 0.38$ for $p \geq 10^5$. Similarly, for the clusters datasets, we use $\lambda = 0.2$ for $p < 10^5$ and $\lambda = 0.23$ for $p \geq 10^5$.

4.1.3 Accuracy with Matrix Tuning Parameter. In this set of tests, we analyze the potential accuracy benefits of using the matrix tuning parameter Λ in a scenario where we have some *a priori* knowledge of the nonzero pattern of the true precision matrix Θ^* . As defined in Equation (7), Λ is constructed using $\mathbf{M}$, where $\Lambda_{ij} = \mathbf{M}_{ij}$ for all nonzero values in $\mathbf{M}$, and $\Lambda_{ij} = \lambda$ otherwise. For a given *bias parameter* $\eta \in (0, \lambda]$, we assign $\mathbf{M}_{ij} = \eta$ for all its nonzero entries, which, for the purposes of our analysis, are configured to be the *corrupted* nonzero structure of Θ^* . We delve further into this construction of $\mathbf{M}$ in the paragraphs that follow. The smaller the ratio η/λ is, the more we induce a bias in the estimated precision matrix $\hat{\Theta}$ to adopt the nonzero pattern of $\mathbf{M}$. At one extreme, for $\eta \rightarrow 0$, we induce the maximum degree of bias as we fix the nonzero pattern of the estimator to that of $\mathbf{M}$. At the other extreme of $\eta = \lambda$, no bias is imposed, and the nonzero structure of $\mathbf{M}$ is not enforced at all, as all entries share the same tuning parameter. This scenario is identical to the scalar tuning parameter where we have that $\Lambda_{ij} = \lambda$.

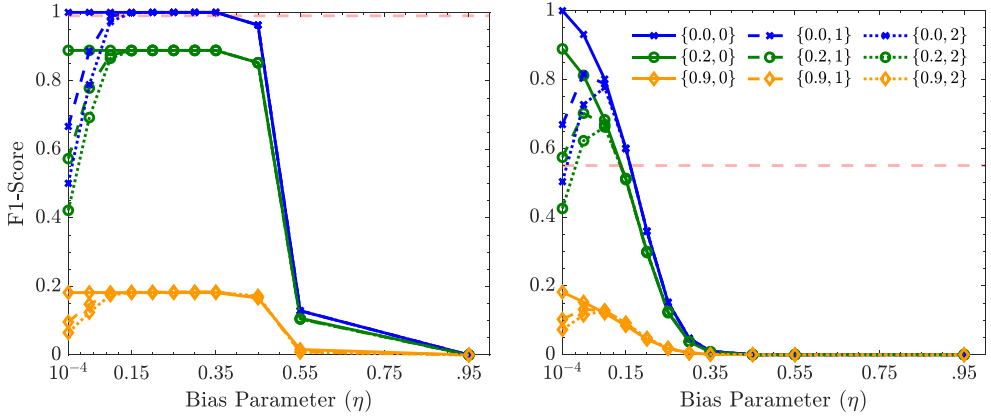


Fig. 4. The F1-Scores using matrix tuning parameters Λ with respect to the bias parameter η for (left) the tridiagonal and (right) the clusters dataset. The test parameters are denoted as $\{b, c\}$, where b and c represent the proportions of FNs and FPs relative to the total number of nonzeros in Θ^* , respectively. The horizontal red dashed line corresponds to the maximum F1-Score achieved using the optimal scalar tuning parameter from the previous section (see Figure 3 for details). Note that for $\eta = 10^{-4}$, the F1-Score accuracies are the same regardless of the dataset.

As denoted in Equation (12), the F1-Score is a function of FNs, which correspond to situations where nonzero entries are erroneously estimated as zeros, and FPs, which refer to instances where zero entries are inaccurately predicted as nonzero.³ We will reference FNs and FPs in relation to s , which denotes the number of nonzeros in the strictly upper-triangular part of Θ^* . For some $b \in [0, 1]$ and $c \geq 0$, we define false negatives and false positives as $\text{FN} = b \cdot s$ and $\text{FP} = c \cdot s$, respectively. Here, b represents the proportion of nonzero elements that have been *missed*, and c is the proportion of incorrectly identified nonzero elements. We can now express Equation (12) as

$$\text{F1-Score} = [1 + 0.5(c + b)/(1 - b)]^{-1}. \quad (13)$$

As long as $b \neq 1$, which means we have successfully identified at least some of the true nonzeros, such as the most significant ones, the F1-Score will improve as both c and b decrease. However, if $b \approx 1$, the F1-Score will be poor and largely insensitive to a decrease in c . This observation is supported by the numerical experiments described next.

In Figure 4, we show the F1-Score of $\hat{\Theta}$ with respect to increasing values of η , with the left panel relating to the tridiagonal dataset and the right one to the clusters dataset. The horizontal red line represents the maximum F1-Score achieved with a scalar tuning parameter λ (further details can be found in Section 4.1.2). In this experiments we fix $\lambda = 0.95$, and both datasets are of size $p = 2 \cdot 10^3$. For each dataset we conduct simulations considering different levels of prior information about the nonzero structure of Θ^* and evaluate the F1-Score accordingly. Each test encodes $\mathbf{M}$ with FPs and FNs using pairwise combinations of $b = 0.0, 0.2$, and 0.9 , and $c = 0, 1$, and 2 , for a total of nine tests per dataset. For example, in a test with $b = 0.2$ and $c = 2$, $\mathbf{M}$ is defined such that it contains 20% of the true nonzeros of Θ^* , with the remaining set erroneously to zero, and twice the number of nonzeros as in the true count. In a situation where $\eta \rightarrow 0$ and λ is sufficiently large, we expect

³Improvements in the F1-Score do not automatically guarantee improvements in the end application results. For example, in a scenario where the nonzero entries of $\hat{\Theta}$ are significantly smaller in absolute value compared to the diagonal entries, it may be sufficient for $\hat{\Theta}$ to be approximated as a diagonal matrix. In such cases, the resulting F1-Score would be poor.

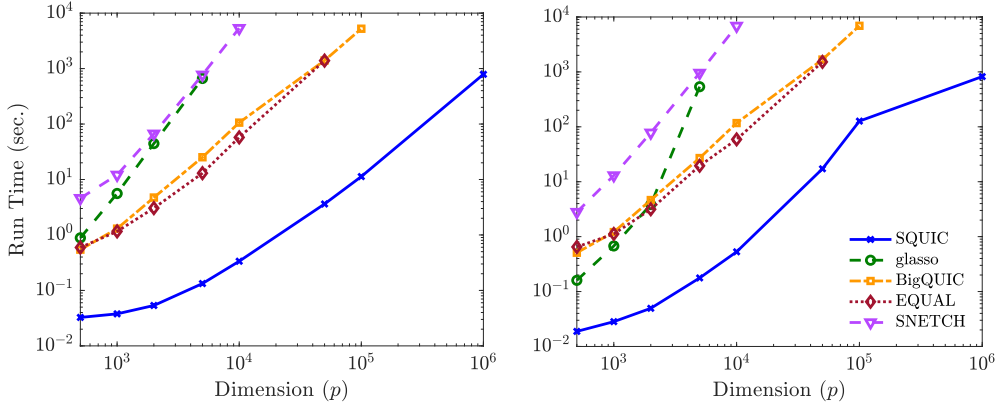


Fig. 5. A comparison of the runtimes for various precision matrix estimation packages, applied to (left) the tridiagonal and (right) the clusters datasets of increasing dimensions p . For all packages with shared-memory parallelization, we have utilized the maximum number of available cores.

$\hat{\Theta}$ to have a nonzero structure identical to that of $\mathbf{M}$. Consequently, irrespective of the dataset, the F1-Scores are exactly those computed by Equation (13) for the predefined values of b and c used in defining $\mathbf{M}$. The observation is confirmed in both plots, with both datasets exhibiting the same F1-Score for $\eta = 10^{-4}$ as computed from Equation (13). As we progressively increase η , it influences both the values of b and c . In particular, an increase in η tends to reduce FPs, effectively reducing c . On the other hand, an excessive increase in η may inadvertently amplify FNs, thereby increasing b . This tradeoff dynamic is clearly evident as the *hump* in both plots across tests for $c > 0$. However, for $c = 0$ we do not have FPs, and thus any increase in η only leads to an increase in b , resulting in a decrease in the F1-Score. When comparing the peak F1-Score using the scalar tuning parameter, we can see that for the tridiagonal dataset, we achieve marginally higher accuracies, with an F1-Score of 1 versus 0.99 for the scalar tuning parameter. We achieve this for all c values but only when $b = 0$. For the clusters dataset, the advantage of using a matrix as the tuning parameter over a scalar is more pronounced with higher F1-Score even when $b = 0.2$. However, in the case where $b = 0.9$, the matrix tuning parameter yields a poor F1-Score. We can anticipate that using $\mathbf{M}$ is most suitable in scenarios where we are relatively confident that we have identified the indices of the salient nonzero values of Θ^* (corresponding to a small value of b). Furthermore, overestimating the number of nonzeros can be mitigated with larger values of η . That being said, the potential accuracy benefits of using the matrix tuning parameter depend not only on selecting the right nonzero structure but also on the dataset and the chosen accuracy measure. In Section 4.2, we will utilize the matrix tuning parameter in a classification example using real-world datasets.

4.1.4 Performance. The runtimes for different precision matrix estimation packages using the tridiagonal and cluster datasets are shown in the left and right panel of Figure 5, respectively. The tests are conducted on a range of dimensions, from $p = 5 \cdot 10^2$ to 10^6 . The number of nonzeros in the estimated precision matrices in these tests is roughly 3 for all tridiagonal datasets and ranges from 4 to 10 in the clusters datasets. In both test cases, SQUIC exhibits significantly faster runtimes across all values of p with a difference of more than an order of magnitude compared to any of the other packages.⁴ For both datasets of size $p \geq 10^4$, the limitations of the competing packages

⁴Note that for SNETCH only the sequential version of the algorithm was available at the time of testing.

Table 1. The Specifications of the Microarray Datasets under Consideration

Name	DATASET			NUMBER OF ITERATIONS	
	K	p	n	Scalar Tuning Parameter	Matrix Tuning Parameter
burczynski	3	22,283	90	7	13
yeoh	6	12,625	175	5	8
shipp	2	7,129	54	6	16
alon	2	2,000	44	6	19

We report the name, the number of ground-truth classes K , the dimension p , and the number of samples n , see [43] for details. Also shown is the number of iterations required by the SQUIC package for convergence when using both a scalar and a matrix tuning parameter.

become clear as the runtimes of SQUIC are on the order of seconds, while most other packages take minutes. For datasets of size $p = 10^5$, the execution of most packages becomes impractical due to their runtimes exceeding an hour; in contrast, SQUIC finishes in less than 2 minutes for both datasets. In the largest case, where $p = 10^6$, none of the competing algorithms finished within 12 hours, whereas SQUIC completed the estimation in less than 15 minutes for both datasets. It is important to note that the performance of SQUIC is highly sensitive to the degree of sparsity in $\hat{\Theta}$ and the intermediate computations involved. In scenarios where the underlying computation has reduced sparsity, SQUIC may exhibit increased runtimes; however, the implemented blocking approach discussed in Section 3 mitigates these effects to some extent.

4.2 Microarray Classification with Matrix Tuning Parameter

In this section, we use **SQUIC for LDA classification** of high-dimensional microarray data pertaining to gene expressions. The study aims to (1) showcase the performance of the introduced package in estimating precision matrices from high-dimensional real-world datasets and (2) highlight the potential for improved classification accuracy through the use of the matrix tuning parameter Λ . The datasets considered are a subset of those available in the datamicroarray package [43] and are listed in Table 1. We begin by loading the data and employing a stratified approach, selecting 70% of the total samples for training and the remaining for testing. Similar to the synthetic datasets, **we normalize the training data to achieve unit variance. This normalization aids in the selection of the tuning parameter and is undone after invoking SQUIC, once the estimation procedure is completed.** To define $\mathbf{M}$ and in turn Λ , we use graph learning methods commonly used in clustering and classification applications (see, e.g., [51] for a survey of various methods). We compute the adjacency matrix $\mathbf{G} \in \mathbb{R}^{p \times p}$ of the **Minimal Spanning Tree (MST)** [41] for the complete graph.⁵ The edge weights of the complete graph are defined as the Euclidean distance (see, e.g., [24] for a similar approach).⁶ This operation is efficiently performed using the external `emstreeR` package [42]. Utilizing $\mathbf{G}$, we define $\mathbf{M}$ with its nonzero pattern and include diagonal entries.⁷ The MST provides a very sparse estimate of $\mathbf{G}$, whose nonzero pattern corresponds to nodes with low Euclidean distance between them, and thus a high similarity (see, e.g., [13] for a detailed discussion).

For comparison purposes, two versions of $\hat{\Theta}$ are computed: one using the matrix tuning parameter and the other using just the scalar tuning parameter. We define Λ by using $\mathbf{M}$ and setting $\eta = 0.1$, and $\lambda = 0.95$. Notice that both η and λ are kept constant for all tests using the matrix tuning

⁵The adjacency matrix $\mathbf{G}$ describes the connectivity of the graph, where $\mathbf{G}_{ij} = 1$ if the nodes (i, j) are connected, or $\mathbf{G}_{ij} = 0$ otherwise.

⁶Using Pearson's distance (refer to [4, 47] for relevant discussions) yields similar accuracy results for the datasets under consideration.

⁷In our estimates, we observe that $\mathbf{G}$ has roughly two nonzeros per row.

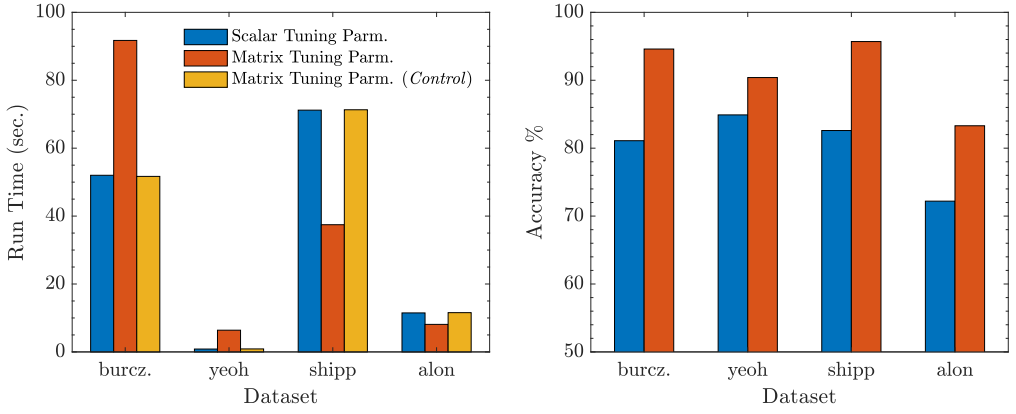


Fig. 6. A comparison of (left) the runtime of SQUIC using scalar and matrix tuning parameters, and (right) the classification accuracy. The *control* test on the left panel utilizes $\eta = \lambda$, resulting in numerical results identical to those of the scalar tuning parameter but incurs a marginal computational overhead. The datasets used correspond to those outlined in Table 1. Note that the runtimes shown here are for the computation of $\hat{\Theta}$ using SQUIC and do not include the creation of $\mathbf{M}$.

parameter. For the scalar tuning parameter case, where $\mathbf{M}$ is not provided, the values of λ which provide the best results have been determined to be 0.7, 0.8, 0.8, and 0.8 for the burczynski, yeoh, shipp, and alon datasets, respectively. **After estimating $\hat{\Theta}$, we undo the normalization by rescaling the estimated precision matrix $\hat{\Theta}$.** The inversion and convergence tolerances are 10^{-3} ; tighter tolerances yielded no numerical improvements in these tests.

We utilize LDA to classify samples from the different test dataset (see, e.g., [8] for a comprehensive review of LDA). Each dataset comprises K classes, where $k = 1, 2, \dots, K$ represents the class label of each individual sample. Let $\hat{\mu}_k$ denote the sample mean for a given class k and $\hat{\Theta}$ be the estimated precision matrix shared by all classes. The discriminant function for class k is

$$\rho_k(\mathbf{x}) = \mathbf{x}^\top \hat{\Theta} \hat{\mu}_k - \frac{1}{2} \hat{\mu}_k^\top \hat{\Theta} \hat{\mu}_k + \log \hat{\pi}_k = \left(\mathbf{x} - \frac{1}{2} \hat{\mu}_k \right)^\top \hat{\Theta} \hat{\mu}_k + \log \hat{\pi}_k, \quad (14)$$

where $\hat{\pi}_k = n_k/n$ is referred to as the *prior* and n_k is the number of samples in class k . The higher the score, the more likely it is that $\mathbf{x}$ belongs to class k . As such, we define the classification of sample $\mathbf{x}$ as

$$C(\mathbf{x}) := \arg \max_k \left\{ \rho_k(\mathbf{x}) \right\}. \quad (15)$$

We perform training and testing for each k . In the training phase, we compute $\hat{\pi}_k$, $\hat{\mu}_k$, and later $\hat{\Theta}$, using the training portion of the dataset. We compute $\hat{\Theta}$ using SQUIC, employing both scalar and matrix tuning parameters. Finally, in the testing phase, we compute ρ_k for all samples in the test datasets. Multiple packages have been used in the implementation of this experiment, the details of which, including code snippets, are outlined in the user manual [18].

Figure 6 showcases the performance of an LDA classification example where the left panel indicates runtime and the right panel demonstrates accuracy. The number of iterations of the algorithm are shown in Table 1. The test denoted with *control* uses $\eta = \lambda$. The control represents an identical optimization problem to that of the scalar tuning parameter but employs the additional data structures required for the matrix tuning parameter. By comparing the runtime of the control with the tests that utilize the scalar tuning parameter, as shown in the left panel of Figure 6, we can see that the implementation of the matrix tuning parameter does not introduce significant

computational overhead. That said, we can also observe a noticeable change in both the number of iterations and overall runtimes when comparing the scalar and matrix tuning parameters. This is expected as a change in the tuning parameter, whether scalar or matrix, alters the optimization problem and, in turn, leads to changes in the intermediate computations. From our experiments, we observe that **when using the matrix tuning parameter, SQUIC requires more iterations for convergence. However, the time per iteration is not necessarily higher.** Only for sufficiently large-scale problems, such as the burczynski dataset with $p \sim 2 \cdot 10^4$, we observed that the change in the runtime of SQUIC is primarily due to an increase in the number of iterations required for convergence. Notice that this is not always the case; for example, the shipp dataset has a lower overall runtime but a higher number of iterations. With regard to classification accuracy, based on these examples, we see that using the matrix tuning parameter can be advantageous. On average, we note a 14% increase in classification accuracies when employing matrix tuning parameters instead of their scalar counterparts.

5 Conclusion

In this article, we have introduced SQUIC, a performant and scalable package for sparse precision matrix estimation. Based on the previous works [6, 16], and [19], this package provides second-order convergence rates in conjunction with highly optimized and parallelized internal subroutines. The core SQUIC algorithm is implemented in C++ with interface packages for R and Python. Our numerical results on synthetic and real-world datasets show that this package is a fitting choice for sparse precision matrix estimation for high-dimensional datasets. Moreover, as demonstrated by both synthetic and real-world experiments, the added flexibility of utilizing a matrix tuning parameter enables the integration of auxiliary information, which can enhance the application results.

Acknowledgments

The authors wish to extend their sincere gratitude to Cristian Ruiz and Emmanuel Jeanvoine for their valued input during the software development phase. Additionally, the authors highly appreciate the insightful comments and inputs provided by the editors and referees.

References

- [1] Hirotugu Akaike. 1992. *Information Theory and an Extension of the Maximum Likelihood Principle*. Springer, New York, NY, 610–624. DOI: https://doi.org/10.1007/978-1-4612-0919-5_38
- [2] Jonas Ballani and Daniel Kressner. 2014. *Sparse Inverse Covariance Estimation with Hierarchical Matrices*. Technical Report. EPFL Technical Report. Retrieved from https://sma.epfl.ch/~anchpcommon/publications/quic_ballani_kressner_2014.pdf
- [3] Onureena Banerjee, Laurent El Ghaoui, and Alexandre d’Aspremont. 2008. Model selection through sparse maximum likelihood estimation for multivariate Gaussian or binary data. *The Journal of Machine Learning Research* 9 (2008), 485–516. Retrieved from <http://www.jmlr.org/papers/volume9/banerjee08a/banerjee08a.pdf>
- [4] Michael R. Berthold and Frank Höppner. 2016. On clustering time series using Euclidean distance and Pearson correlation. Retrieved from <https://arxiv.org/abs/1601.02213>
- [5] Dimitris Bertsimas, Jourdain Lamperski, and Jean Pauphilet. 2020. Certifiably optimal sparse inverse covariance estimation. *Mathematical Programming* 184, 1 (2020), 491–530. DOI: <https://doi.org/10.1007/s10107-019-01419-7>
- [6] Matthias Bollhöfer, Aryan Eftekhari, Simon Scheidegger, and Olaf Schenk. 2019. Large-scale Sparse Inverse Covariance Matrix Estimation. *SIAM Journal on Scientific Computing* 41, 1 (Jan 2019), A380–A401. DOI: <https://doi.org/10.1137/17M1147615>
- [7] Tony Cai, Hongzhe Li, Weidong Liu, and Jichun Xie. 2012. Covariate-adjusted precision matrix estimation with an application in genetical genomics. *Biometrika* 100, 1 (November 2012), 139–156. DOI: <https://doi.org/10.1093/biomet/as058>
- [8] Daniela Calvetti and Erkki Somersalo. 2020. Linear discriminant analysis. In *Mathematics of Data Science: A Computational Approach to Clustering and Classification*. SIAM. Chapter 4, 49–61. Retrieved from <https://epubs.siam.org/doi/abs/10.1137/1.9781611976373.ch4>

- [9] Yanqing Chen, Timothy A. Davis, William W. Hager, Sivasankaran Rajamanickam, and W. W. Hager. 2008. Algorithm 887: CHOLMOD, supernodal sparse Cholesky factorization and update/downdate. *ACM Transactions on Mathematical Software (TOMS)* 35, 14 (2008), 1–14. DOI: <https://doi.org/10.1145/1391989.1391995>
- [10] L. Dagum and R. Menon. 1998. OpenMP: An industry standard API for shared-memory programming. *IEEE Computational Science and Engineering* 5, 1 (1998), 46–55. DOI: <https://doi.org/10.1109/99.660313>
- [11] Hercules Dalianis. 2018. *Evaluation Metrics and Evaluation*. Springer International Publishing, Cham, 45–53. DOI: https://doi.org/10.1007/978-3-319-78503-5_6
- [12] Timothy A. Davis, Sivasankaran Rajamanickam, and Wissam M. Sid-Lakhdar. 2016. A survey of direct methods for sparse linear systems. *Acta Numerica* 25 (2016), 383–566. DOI: <https://doi.org/10.1017/S0962492916000076>
- [13] Patrik D’haeseleer. 2005. How does gene expression clustering work? *Nature Biotechnology* 23, 12 (2005), 1499–1501. DOI: <https://doi.org/10.1038/nbt1205-1499>
- [14] Bradley Efron. 1982. *The Jackknife, the Bootstrap and Other Resampling Plans*. Society for Industrial and Applied Mathematics. DOI: <https://doi.org/10.1137/1.9781611970319>
- [15] Aryan Eftekhari. 2021. *Scalable Algorithms for High-Dimensional Graphical Lasso and Function Approximation*. Ph.D. Dissertation. Università della Svizzera Italiana, Lugano. Retrieved from <https://susi.usi.ch/usi/documents/319150>
- [16] Aryan Eftekhari, Matthias Bollhöfer, and Olaf Schenk. 2018. Distributed memory sparse inverse covariance matrix estimation on high-performance computing architectures. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis (SC ’18)*. IEEE Press, Dallas, TX. Article 20, 12 pages. DOI: <http://doi.org/10.1109/SC.2018.00023>
- [17] Aryan Eftekhari, Lisa Gaedke-Merzhäuser, Dimosthenis Pasadakis, Matthias Bollhöfer, Simon Scheidegger, and Olaf Schenk. 2023a. SQUIC Project Repository. Retrieved from <https://www.gitlab.ci.inf.usi.ch/SQUIC>.
- [18] Aryan Eftekhari, Lisa Gaedke-Merzhäuser, Dimosthenis Pasadakis, Matthias Bollhöfer, Simon Scheidegger, and Olaf Schenk. 2023b. SQUIC User Manual, Software Package for Sparse Precision Matrix Estimation. Retrieved from https://www.gitlab.ci.inf.usi.ch/SQUIC/gitlab-profile/-/raw/main/SQUIC_User_Manual.pdf
- [19] Aryan Eftekhari, Dimosthenis Pasadakis, Matthias Bollhöfer, Simon Scheidegger, and Olaf Schenk. 2021. Block-enhanced precision matrix estimation for large-scale datasets. *Journal of Computational Science* 53 (2021), 101389. DOI: <https://doi.org/10.1016/j.jocs.2021.101389>
- [20] Jianqing Fan, Yingying Fan, and Jinchi Lv. 2008. High dimensional covariance matrix estimation using a factor model. *Journal of Econometrics* 147 (2008), 186–197. DOI: <https://doi.org/10.1016/j.jeconom.2008.09.017>
- [21] Jianqing Fan, Yuan Liao, and Han Liu. 2016. An overview of the estimation of large covariance and precision matrices. *The Econometrics Journal* 19, 1 (2016), C1–C32. DOI: <https://doi.org/10.1111/ectj.12061>
- [22] Jerome Friedman, Trevor Hastie, and Robert Tibshirani. 2007. Sparse inverse covariance estimation with the graphical lasso. *Biostatistics* 9, 3 (December 2007), 432–441. DOI: <https://doi.org/10.1093/biostatistics/kxm045>
- [23] Jerome Friedman, Trevor Hastie, and Rob Tibshirani. 2019. *glasso: Graphical Lasso: Estimation of Gaussian Graphical Models*. R package version 1.11. Retrieved from <https://CRAN.R-project.org/package=glasso>
- [24] Francis D. Gibbons and Frederick P. Roth. 2002. Judging the quality of gene expression-based clustering methods using gene annotation. *Genome Research* 12, 10 (October 2002), 1574–1581. DOI: <https://doi.org/10.1101/gr.397002>
- [25] Maarten Gulliksson and Stepan Mazur. 2020. An iterative approach to ill-conditioned optimal portfolio selection. *Computational Economics* 56, 4 (2020), 773–794. DOI: <https://doi.org/10.1007/s10614-019-09943-6>
- [26] Cho-Jui Hsieh, Inderjit Dhillon, Pradeep Ravikumar, and Mátyás Sustik. 2011. Sparse inverse covariance matrix estimation using quadratic approximation. In *Proceedings of the Advances in Neural Information Processing Systems*. J. Shawe-Taylor, R. Zemel, P. Bartlett, F. Pereira, and K. Q. Weinberger (Eds.), Vol. 24. Curran Associates, Inc. Retrieved from <https://proceedings.neurips.cc/paper/2011/file/2ba8698b79439589fdd2b0f7218d8b07-Paper.pdf>
- [27] Cho-Jui Hsieh, Matyas A. Sustik, Inderjit S. Dhillon, Pradeep K. Ravikumar, and Russell Poldrack. 2013. BIG & QUIC: Sparse inverse covariance estimation for a million variables. In *Proceedings of the Advances in Neural Information Processing Systems*. C. J. C. Burges, L. Bottou, M. Welling, Z. Ghahramani, and K. Q. Weinberger (Eds.), Vol. 26. Curran Associates, Inc. Retrieved from <https://proceedings.neurips.cc/paper/2013/file/1abb1e1ea5f481b589da52303b091cbb-Paper.pdf>
- [28] Anil K. Jain, Robert P. W. Duin, and Jianchang Mao. 2000. Statistical pattern recognition: A review. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 22, 1 (2000), 4–37. DOI: <https://doi.org/10.1109/34.824819>
- [29] Vladislav Jelisavcic, Ivan Stojkovic, Veljko Milutinovic, and Zoran Obradovic. 2018. Fast learning of scale-free networks based on Cholesky factorization. *International Journal of Intelligent Systems* 33, 6 (2018), 1322–1339. DOI: <https://doi.org/10.1002/int.21984>
- [30] Lieven Vandenbergh, Joachim Dahl and Vwani Roychowdhury. 2008. Covariance selection for nonchordal graphs via chordal embedding. *Optimization Methods and Software* 23, 4 (2008), 501–520. DOI: <https://doi.org/10.1080/10556780802102693>

- [31] George Karypis and Vipin Kumar. 1998. METIS: A Software Package for Partitioning Unstructured Graphs, Partitioning Meshes, and Computing Fill-Reducing Orderings of Sparse Matrices. Retrieved from <https://conservancy.umn.edu/handle/11299/215346>
- [32] Markku O. Kuusimäki and Mikko J. Sillanpää. 2017. Estimation of covariance and precision matrix, network structure, and a view toward systems biology. *Wiley Interdisciplinary Reviews: Computational Statistics* 9, 6 (November 2017), e1415. DOI: <https://doi.org/10.1002/wics.1415>
- [33] Khalid B. Kunji. 2020. BigQuic: Big Quadratic Inverse Covariance Estimation. R package version 1.1-9. Retrieved from <https://CRAN.R-project.org/package=BigQuic>
- [34] Olivier Ledoit and Michael Wolf. 2003. Improved estimation of the covariance matrix of stock returns with an application to portfolio selection. *Journal of Empirical Finance* 10, 5 (2003), 603–621. Retrieved from <https://EconPapers.repec.org/RePEc:eee:empfin:v:10:y:2003:i:5:p:603-621>
- [35] Jason D. Lee, Yuekai Sun, and Michael A. Saunders. 2014. Proximal Newton-type methods for minimizing composite functions. *SIAM Journal on Optimization* 24, 3 (2014), 1420–1443. DOI: <https://doi.org/10.1137/130921428>
- [36] Han Liu, Kathryn Roeder, and Larry Wasserman. 2010. Stability approach to regularization selection (StARS) for high dimensional graphical models. In *Proceedings of the 23rd International Conference on Neural Information Processing Systems (NIPS'10)*, Vol. 2, Vancouver, BC, Canada. Curran Associates Inc., Red Hook, NY, 1432–1440.
- [37] Joseph W. H. Liu, Esmond G. Ng, and Barry W. Peyton. 1993. On finding supernodes for sparse matrix computations. *SIAM Journal on Matrix Analysis and Applications* 14, 1 (1993), 242–252. DOI: <https://doi.org/10.1137/0614019>
- [38] Geoffrey J. McLachlan. 1992. *Discriminant Analysis and Statistical Pattern Recognition*. Wiley, 526 pages.
- [39] Figen Oztoprak, Jorge Nocedal, Steven Rennie, and Peder A. Olsen. 2012. Newton-like methods for sparse inverse covariance estimation. In *Proceedings of the Advances in Neural Information Processing Systems*, Vol. 25. Retrieved from https://papers.nips.cc/paper_files/paper/2012/file/b3967a0e938dc2a6340e258630feb5a-Paper.pdf
- [40] Dimosthenis Pasadakis, Matthias Bollhöfer, and Olaf Schenk. 2023. Sparse quadratic approximation for graph learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45, 9 (2023), 11256–11269. DOI: <https://doi.org/10.1109/TPAMI.2023.3263969>
- [41] Robert Clay Prim. 1957. Shortest connection networks and some generalizations. *The Bell System Technical Journal* 36, 6 (1957), 1389–1401. DOI: <https://doi.org/10.1002/j.1538-7305.1957.tb01515.x>
- [42] Allan Quadros. 2022. emstreeR: Tools for Fast Computing and Plotting Euclidean Minimum Spanning Trees. R package version 3.0.0. Retrieved from <https://CRAN.R-project.org/package=emstreeR>
- [43] John A. Ramey. 2016. datamicroarray: Collection of Data Sets for Classification. Retrieved from <https://github.com/ramhiser/datamicroarray>
- [44] Benjamin Rolfs, Bala Rajaratnam, Dominique Guillot, Ian Wong, and Arian Maleki. 2012. Iterative thresholding algorithm for sparse inverse covariance estimation. In *Proceedings of the Advances in Neural Information Processing Systems*. F. Pereira, C. J. Burges, L. Bottou, and K. Q. Weinberger (Eds.), Vol. 25. Curran Associates, Inc. Retrieved from https://proceedings.neurips.cc/paper_files/paper/2012/file/edf8e1afcf9246bb0d40eb4d8027d90f-Paper.pdf
- [45] Havard Rue and Leonhard Held. 2005. *Gaussian Markov Random Fields: Theory and Applications (Monographs on Statistics and Applied Probability)*. Chapman & Hall/CRC. DOI: <https://doi.org/10.1201/9780203492024>
- [46] Gideon Schwarz. 1978. Estimating the dimension of a model. *The Annals of Statistics* 6, 2 (1978), 461–464. Retrieved from <http://www.jstor.org/stable/2958889>
- [47] Victor Solo. 2019. Pearson distance is not a distance. Retrieved from <https://arxiv.org/abs/1908.06029>
- [48] Janet Van Niekerk, Haakon Bakka, Haavard Rue, and Olaf Schenk. 2021. New frontiers in Bayesian modeling using the INLA package in R. *Journal of Statistical Software* 100, 2 (2021), 1–28. DOI: <https://doi.org/10.18637/jss.v100.i02>
- [49] Cheng Wang and Binyan Jiang. 2019. EQUAL: An efficient ADMM algorithm for high dimensional precision matrix estimation via penalized quadratic loss. R package version 1.2. Retrieved from <https://github.com/cescwang85/EQUAL>
- [50] Cheng Wang and Binyan Jiang. 2020. An efficient ADMM algorithm for high dimensional precision matrix estimation via penalized quadratic loss. *Computational Statistics & Data Analysis* 142 (2020), 106812. DOI: <https://doi.org/10.1016/j.csda.2019.106812>
- [51] Feng Xia, Ke Sun, Shuo Yu, Abdul Aziz, Liangtian Wan, Shirui Pan, and Huan Liu. 2021. Graph learning: A survey. *IEEE Transactions on Artificial Intelligence* 2, 2 (2021), 109–127. DOI: <http://doi.org/10.1109/TAL.2021.3076021>
- [52] Jieping Ye and Jun Liu. 2012. Sparse methods for biomedical data. *SIGKDD Explorations: Newsletter of the Special Interest Group (SIG) on Knowledge Discovery & Data Mining* 14, 1 (June 2012), 4–15. DOI: <https://doi.org/10.1145/2408736.2408739>
- [53] Ming Yuan and Yi Lin. 2007. Model selection and estimation in the Gaussian graphical model. *Biometrika* 94, 1 (2007), 19–35. DOI: <https://doi.org/10.1093/biomet/asm018>

Received 1 September 2022; revised 11 December 2023; accepted 22 February 2024